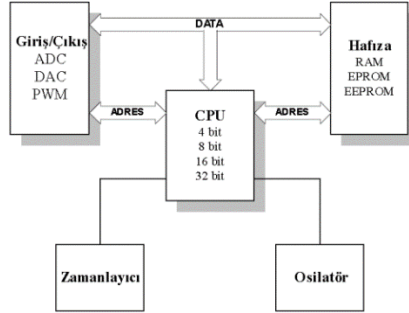
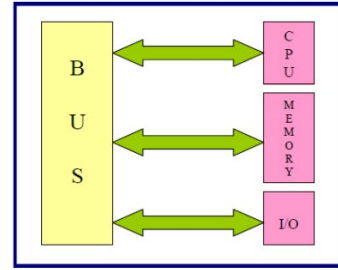
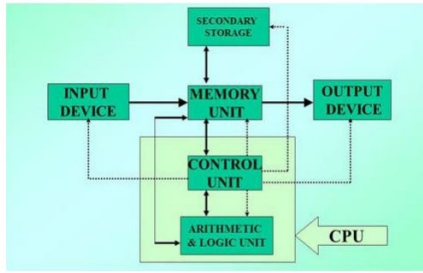


MİKROİŞLEMCİLER

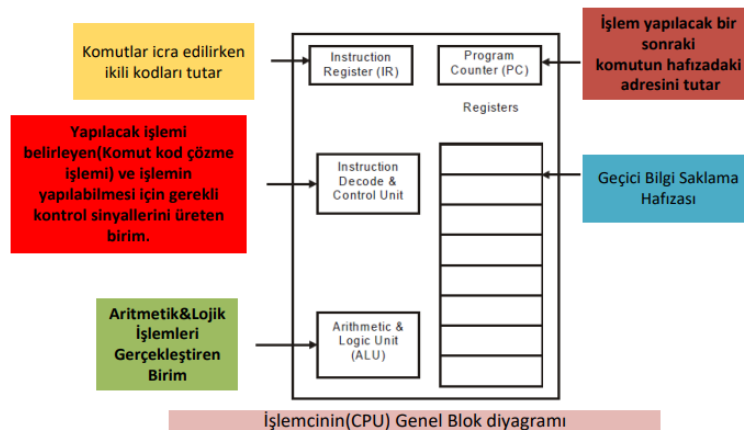
- Mikroişlemcili sistemlerin genel yapısı aşağıdaki gibidir:



- Osilatör:** Kare dalga üreticidir. İşlemcinin hızını etkiler. (GHz, MHz vs.) **Görevi;** veri ve komutların CPU'ya alınmasında, yürütülmesinde, kayıt edilmesinde, sonuçların hesaplanmasında ve çıktıların ilgili birimlere gönderilmesinde gerekli olan saat darbelerini üretmektir.
- Mikroişlemcili sistemler ile mikrodenetleyici farkı:**
 - Mikroişlemciler; CPU'dan oluşan bir entegre devredir.
 - Mikrodenetleyiciler; RAM, ROM, I/O birimi ve CPU'dan oluşan bir entegre devredir.



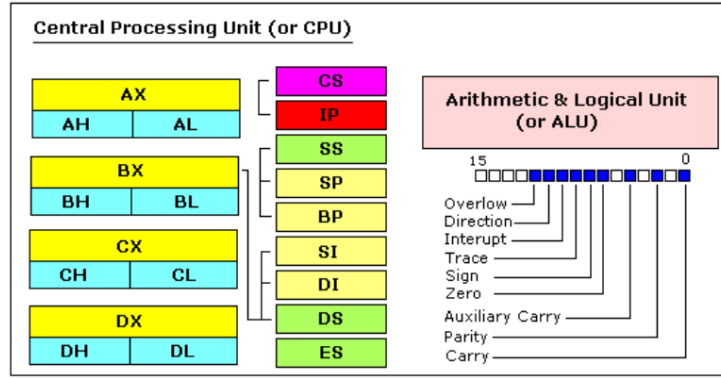
- Mikrodenetleyici** donanıma daha yakındır. Normalde mikrodenetleyici kullanırken, donanımın tüm bilgisine sahip olmak gerekir. Fakat günümüzde, Arduino gibi sistemlerle, donanımdan bağımsızlaştırılmış mikrodenetleyiciler kullanılmaktadır.
- Mikroişlemci,** program sayacına sahiptir. Program sayacı, programın nerede kaldığını tutan sayaçtır.
- Merkezi İşlem Birimi (CPU):** Bilgisayar ve Mikrodenetleyicilerin beynidir. Sistemdeki bütün aktiviteleri kontrol eder ve veri üzerindeki bütün işlemleri yapan birimdir. Sürekli olarak "Makine Döngüsü" de denilen, şu işlemleri gerçekleştirir: Komutları Gidip-Getirme (**Fetch**), Komutları Çözme (**Decode**), Komutu İşleme (**Execute**) ve sonucun bir çıkışa gönderilmesi veya bir yere yazılması (**Store**) işlemi de gerçekleştirilir. Komut kümesi denen ikili kodlardan oluşan komutları anlar ve istenen işlemi gerçekleştirir.



- Bir makine döngüsü = X clock (saat) darbesi** (X, komuta bağlı olarak değişir ve işlem süresini belirler).
- Anayol (BUS):** Bir amaç doğrultusunda veri akışını sağlayan yapılardır. Örnek olarak okuma/yazma işlemi sırasında CPU işlem yapacağı veri adresini ADRES hattına yerleştirir ardından KONTROL anayolu üzerinden okuma veya yazma sinyalini aktif eder.
- Adres Anayolu:** Belirlenen adres bilgisini taşır. Eğer n adres hattı varsa 2ⁿ adres tanımlanabilir. 16 bit adres hattı varsa 2¹⁶ = 65536 = 64K adres tanımlanabilir.
- Veri Anayolu:** CPU ile Bellek arasında veya CPU ile I/O cihazları arasında veri iletişimini sağlar.
- KONTROL Anayolu:** Adres ve Veri Anayolu üzerinde taşınan verinin senkronizasyonunu sağlamakla sorumludur.
- Bit:** Bilgi olarak saklanabilen en küçük birimdir.

- **Byte:** 8 bitten oluşan birim.
- **Nibble:** 4 bit'ten oluşan birimdir. Yüksek nibble veya düşük nibble olarak kullanılır.
- **Word:** Bilgisayara bağlı olarak değişen 16 bit, 32 bit veya 64 bitlik gruplardır.
- **Bir Gidip-Getirme ve İşleme Döngüsü Adımları**
 - 1) Program Sayacı (PC) içeriği Adres anayoluna (Bus) yerleştirilir.
 - 2) OKUMA sinyali aktif edilir.
 - 3) Veri (komut opcode) RAM'den okunur ve Veri anayoluna yerleştirilir.
 - 4) Opcode CPU'nun içindeki komut kaydedicisine (register) yüklenir.
 - 5) PC bir sonraki gidip-getirme işlemi için arttırılır. (otomatik olarak artar)
- **Mikroişlemcilerin sınıflandırılması**
 - Anayol genişliklerine göre: ALU'nun sahip olduğu veri anayolunun boyutu (kaç bitlik olduğu) tipini belirler. 4,8,16,32 veya 64bit.
 - Mimariye göre:
 - **Von-Neuman (veya Princeton) mimarisi:** Kod ile Data aynı memory'i kullanır. Daha çok mikrodenetleyicilerde kullanılır.
 - **Harvard Mimarisi:** Daha çok kişisel bilgisayarlarda kullanılır.
 - Komut setlerine göre: CISC, RISC ve SISC.
- **Aritmetik ve Mantık Birimi (ALU):** Bilgisayar biliminde kısaca ALU olarak ifade edilir. Toplama, çıkarma, çarpma, bölme gibi aritmetik işlemler ve AND, OR, XOR, NOT gibi mantık işlemleri gerçekleştirilir.
- **Kontrol Birimi:** Bilgisayarda yapılan tüm işlemleri kontrol eden birimdir. Giriş ve çıkış birimlerinin denetimini, bellek ile ilgili işlemleri, komutların yorumlanması ve bilgisayarın bir bütün olarak çalışmasını sağlar. Kontrol birimi, tüm komutları bellekte bulunış sıralarına göre işlemektedir. Bu sıranın belirlenmesi için program sayacı (program counter) adı verilen yazmaçtan yararlanır.
- **Program sayacı:** İşlemci tarafından işlenen komutun adresini taşıyan yazmaçtır. Komutlar bulundukça, program sayacının değeri 1 arttırılır. Bilgisayarda bir sonraki adımda işlenecek komutun adresi, program sayacında bulunur. Kontrol birimi, her komutu getirdikten sonra program sayacının değerini 1 arttırır. Bilgisayar yeniden başlatıldığında, program sayacının değeri de 0'a çevrilir.
- **Yazmaçlar (Registers):** Kaydedici olarak da adlandırılan yazmaç, mikroişlemciler tarafından kullanılan dâhili geçici hafızalardır.
- **CPU, yalnızca makine dilinde yazılmış komutları algılayabilir.** Bir programda yer alan her komut, merkezî işlem birimi tarafından yapılacak bir işleme denk gelir. CPU'nun yapabileceği işlemlerin makine dilinde karşılığı bulunur ve bu işlemlerin tümünü içeren kümeye komut kümesi (instruction set) adı verilir. Komut kümesi mimarisi, bilgisayar sistemlerinde donanım ve yazılım arasında ayırma katmanı olarak yer alır. **CPU, kendi başına çalışamaz.** Komut listesi içeren programlar tarafından yönlendirilir.
- **Komut kümesi:** Mikroişlemcinin tasarlanma ve üretim aşamalarında tanımlanan, mikroişlemci tarafından algılanabilen komutlar kümesidir. Günümüzde Intel, AMD, Samsung ve IBM gibi işlemci üreten birçok firma vardır. Her firmanın kendine ait bir komut kümesi bulunur. Dolayısıyla, farklı firmaların mikroişlemcileri aynı komutları algılayamayabilir.
- **CISC (Complex Instruction Set Computer):** "Karmaşık Komut Kümeli Bilgisayar" anlamına gelen bu yaklaşımda, işlemcilerin çok sayıda farklı komutu çalıştırabilecek kapasitede olması hedeflenir. Komut çeşitliliği sayesinde, gelişen ve karmaşıklaşan yazılımlar ile daha rahat başa çıkılabilir. Komut kümesinde, temel komutların yanı sıra, teknik olarak çok fazla ihtiyaç duyulmayan komutlar da bulunur. CISC modelini kullanan işlemci üretici firmalar, Intel ve AMD olarak örneklendirilebilir.
- **RISC (Reduced Instruction Set Computer):** "Azaltılmış Komut Kümeli Bilgisayar" anlamına gelen bu yaklaşıma göre, bir işlemcinin komut kümesinde temel komutların olması yeterlidir ve temel komutların çeşitli organizasyonu, her türlü karmaşık işlem yapılabilir. Bellek hızı artırıldığından ve yüksek seviyeli diller Assembly dilinin yerini aldığından, CISC'in başlıca üstünlükleri geçersizleşmeye başladı. Bilgisayar tasarımcıları sadece donanımı hızlandırmaktan çok bilgisayar performansını iyileştirmek için başka yollar denemeye başladılar. Bir RISC işlemcisinin performansı işlediği kodun algoritmasına çok bağlıdır. Eğer programcı veya derleyici, komut programlamada zayıf iş çıkarırsa, işlemci atıl durumda kalarak bir parça zaman harcayabilir.
- **Modern bilgisayarların büyük kısmında CISC yaklaşımını kullanan,** Intel ve AMD işlemcileri tercih edilir. RISC yaklaşımı ile tasarlanmış işlemciler, genellikle daha az elektrik tüketirler. Elektrik ihtiyacı konusundaki bu önemli avantaj, bu yaklaşım ile tasarlanmış işlemcilerin cep telefonlarında, akıllı telefonlarda, dijital televizyonlarda ve navigasyon sistemlerinde kullanılmasını sağlar.
- **Von Neuman (Princeton) Mimarisi:** İlk bilgisayarlar Von Neuman yapısından yola çıkılarak geliştirilmiştir. Geliştirilen bu bilgisayar beş birimden oluşmaktaydı. Bu birimler; ALU, CU, Bellek (RAM ve Registers), G/Ç birimleri ve Veriyoludur. Bu mimaride veri ve komutlar bellekten tek bir yoldan mikroişlemciye getirilerek işlenmektedir. Program ve veri aynı bellekte bulunduğundan, komut ve veri gerekli olduğunda aynı iletişim yolunu kullanmaktadır. Bu mimari, verilerin işlenmesinde, bilginin derlenmesinde ve sayısal problemlerde olduğu kadar endüstriyel denetimlerde kullanılmaktadır.
- **Harvard Mimarisi:** Von Neuman mimarisinden farklı olarak, veri ve komutlar ayrı ayrı belleklerde tutulur. Veri ve komut aktarımında iletişim yolları da birbirinden bağımsız yapıdadır. Bu mimari günümüzde daha çok sayısal sinyal işlemcilerinde (DSP) kullanılmaktadır. Bu mimaride program içerisinde döngüler ve zaman gecikmeleri daha kolay ayarlanır. Von Neuman yapısına göre daha hızlıdır. Özellikle PIC mikrodenetleyicilerinde bu yapı kullanılır.
- **Assembly Dili Komutları dört bölümden oluşur:** 1) Etiket, 2) Mnemonic, 3) Operand, 4) Yorumlar.


```
main: (Etiket)
      MOV AX,5;
      ADD AX,6; 5+6=11 AX'e eklenir.
```
- **8086 işlemcisinin adresleme kapasitesi** 1 MB = 2^{20} bit'tir. Veri yolu ve tüm iç registerlar ise, **16 bit** genişliğindedir.
- **8086 işlemcisinde, en fazla adreslenebilir hafıza uzayı** 64 KB büyüklüğündedir. Çünkü tüm dahili register'lar 16 bit büyüklüğündedir. 64 KB'lık sınırların dışında programlama yapabilmek için extra operasyonlar kullanmak gerekir.



- **Register'lar:** CPU içerisinde bulunduklarından dolayı, hafıza bloğuna (RAM) göre oldukça hızlıdır. Hafıza bloğuna erişim için sistem veri yollarının kullanılması gereklidir. Register'daki verilere ulaşmak için çok küçük bir zaman dilimi yeterli olur. Bu sebeple, değişkenlerin, register'larda tutulmasına çalışmalıdır. Register grupları genellikle oldukça kısıtlıdır ve çoğu register'ın önceden tanımlanmış görevleri bulunur. Bu nedenle, kullanımları çok sınırlıdır. Ancak, yine de hesaplamalarda geçici hafıza birimi olarak kullanmak için en ideal birimlerdir.
- **REGISTER TÜRLERİ**
 - **Genel Amaçlı Register'lar:** 8086 CPU'da, 8 genel amaçlı register bulunur. Her register'ın ayrı bir ismi bulunur. Bu register'lar, isminin belirttiği amaçlar için kullanılmak zorunda değildir. Programcı, genel amaçlı register'ları istediği gibi kullanabilir.
 - **AX - accumulator register** – Akümülatör (AH / AL): Temel register. Birçok aritmetik işlem, bu register üzerinden gerçekleştirilir. İşlemin sonucu sıfır çıkarsa, zero bayrağı set olur.
 - **BX - the base address register** – Adres başlangıcı (BH / BL).
 - **CX - the count register** – Sayma (CH / CL).
 - **DX - the data register** – Veri (DH / DL).
 - **SI - source index register** – Kaynak indisi.
 - **DI - destination index register** – Hedef indisi.
 - **BP - base pointer** – Temel gösterici.
 - **SP - stack pointer** – Yığıt gösterici.
 - **Segment (İndis) Register'ları:** Segment register'larında herhangi bir veriyi depolamak mümkündür. Ancak bu, güzel bir fikir değildir. Segment register'larının özel amaçları vardır. Hafızada ulaşılabilir bazı bölümleri işaretler. Segment register'ları, genel amaçlı register'ları ile birlikte çalışarak hafızada herhangi bir bölgeyi işaretleyebilir. Örneğin, fiziksel adres 12345h (heksadesimal) işaretlenmesi isteniyor ise, DS = 1230h ve SI = 0045h olmalıdır. CPU, segment register'ı 10h ile çarpar ve genel amaçlı register'da bulunan değeri de ilave eder ($1230h \times 10h + 45h = 12345h$).
 - **CS (Code Segment):** Mevcut programın bulunduğu bölümü işaretler.
 - **DS (Data Segment):** Genellikle programda bulunan değişkenlerin bulunduğu bölümü işaretler. **BX, SI, DI** ile birlikte çalışır.
 - **ES (Extra Segment):** Bu register'ın kullanımı, kullanıcıya bırakılmıştır. **SI, DI** ile birlikte çalışır.
 - **SS (Stack Segment):** Yığının (stack) bulunduğu bölümü işaretler. **BP, SP** ile birlikte çalışır.
 - **Özel Amaçlı Register'lar:**
 - **IP (Instruction pointer - komut işaretleyicisi):** IP register'ı CS ile birlikte, halihazırdaki çalıştırılan komutu işaretler.
 - **Flags (Bayrak) register:** Flags register, CPU tarafından, matematiksel operasyonlardan sonra otomatik olarak değiştirilir. Bu register sayesinde, elde edilen sonucun çeşidi ve durumu ile ilgili bilgiler, program tarafından kullanılabilir. Genellikle, bu register'lara doğrudan erişim bulunmaz. Ancak, sistem register'larının değerleri, daha sonra öğreneceğiniz bazı metotlar sayesinde değiştirilebilir.
- **Register'ların ana amacı, bir değişkeni tutmaktır.** Yukarıdaki register'ların tamamı 16 bittir. 4 genel amaçlı register (AX, BX, CX, DX), iki ayrı 8 bitlik register olarak kullanılabilir. Örneğin eğer **AX=3A39h** ise (3A39 verisi 16 bittir), bu durumda **AH=3Ah** ve **AL=39h** olur. 8 bitlik register'ları değiştirdiğiniz zaman, 16-bitlik register'lar da değişmiş olur.
- **Effective address (Efektif Adres):** 2 register tarafından oluşturulmuş olan adrestir. Diğer genel amaçlı register'lar, efektif adres oluşturmak için kullanılmazlar. Ayrıca, BX efektif adres oluşturulmasında kullanılırken, BH ve BL kullanılmaz.
- **Segment ve Offset:** Segment adresi, herhangi bir hafıza bölümünün başlangıcı gösterir. Offset adresi ise, o hafıza bölümündeki herhangi bir satırı belirtir. Tüm hafıza adresleri segment adresine offset adresi ilave edilmesi ile bulunur. Segment ve offset adresleri 1000:2000 biçiminde de yazılabilir. Bu durumda segment adresi 1000h ve offset'te 2000h'dir.
$$\text{Fiziksel Adres} = \text{Segment Adresi} \times 10 + \text{Offset}$$
- **CS:IP** -> CS, kod bölümünün başlangıcına işaret eder. IP, kod bölümü içerisinde bir sonraki komutun bulunduğu hafıza adresine işaret eder. Eğer CS=1400h ve IP=1200h ise mikroişlemci, sonraki komutu $1400h \times 10 + 1200h = 14000h + 1200h = 15200h$ adresinden okur.
- **SS:SP** veya **SS:BP** -> Yığın (stack) bölümünü kullanan komutlar kullanır. **SP; DI, IP** ve **SI** ile birlikte kullanılmaz.
- **Flag Bitleri:** P ve A flag bitleri çok sık kullanılmaz.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
				O	D	I	T	S	Z		A		P		C

- **C (Carry):** Toplama işleminde oluşan elde ve çıkarma işleminde oluşan ödünçleri tutar. Ayrıca, hata durumlarını gösterir.

CARRY FLAG (CF)

İşaretsiz sayılar ile yapılan işlemlerden yaşanan taşmalar sonucunda bu bayrak **aktif** (1) olur. İşlem sonucunda herhangi bir taşma meydana gelmez ise **pasif** (0) olur.

8 bitlik bir ifade için işaretli sayılar **0-255** arasındadır. Örneğin;

255+1 gibi bir toplama işleminde taşma meydana gelir. Çünkü toplama sonucunda ortaya çıkan değer 8 bit ile ifade edilemez.

- **Z (Zero):** Aritmetik veya mantık operasyonunun sonucunun sıfır olup olmadığı bilgisini tutar.

ZERO FLAG (ZF)

Herhangi bir işlemin sonucunda sıfır değeri elde ediliyorsa ZF **aktif** (1) olur. Gerçekleşen işlemin sonucunda sıfır değeri elde edilmiyorsa ZF **pasif** (0) olur.

Örneğin:

4-4 gibi bir işlem yapıldığı zaman sonuç 0 olacağı için ZF aktif olacaktır. ZF aktif olup olmadığı kontrol edilerek iki sayının birbirine eşit olup olmadığı sonucu çıkarılabilir.

- **S (Sign):** Çalıştırılan aritmetik/mantık operasyonunun sonucunda elde edilen sayının aritmetik yönünü (pozitif/negatif) belirtir.

SIGN FLAG (SF)

ALU tarafından gerçekleştirilen bir işlemin sonucu eğer negatif çıkıyorsa işaret bayrağı **aktif** (1) olur. Eğer işlemin sonucunda negatif bir değer oluşmuyorsa SF **pasif** (0) olur.

Örneğin:

4-5 gibi bir işlemin sonucu -1 yani negatif bir sayı olacağı için SF aktif olacaktır. SF kontrol edilerek 1.sayının 2.sayıdan küçük olup olmadığı anlaşılabilir.

- **O (Overflow):** İki yönlü sayının toplamı veya çıkarılması durumlarında kullanılır. Overflow bayrağının 1 olması, sonucun 16-bitlik kapasiteyi aştığını gösterir.

OVERFLOW FLAG (OF)

İşaretli sayılarda işlem sonucu işaretli sayı aralığını aşıyorsa taşma bayrağı **aktif** (1) olacaktır. Eğer sonuç işaretli sayı aralığında ise OF **pasif** (0) olarak ayarlanacaktır.

8 bitlik işaretli sayılar için en küçük değer **-128** en büyük değer **+127**'dir.

Örneğin:

100+50 gibi bir toplama işleminden çıkan sonuç -128 ile +127 aralığı dışında olduğu için OF aktif (1) olacaktır.

100+10 gibi bir toplama işleminde çıkan sonuç -128 ile +127 aralığında olduğu için OF pasif (0) olacaktır.

- **P (Parity):** Bir sayıda bulunan 1'lerin tek sayıda mı? yoksa çift sayıda mı? olduğunu belirtir. P=0 tek parity; P=1 çift parity anlamına gelir. Eğer binary bir sayı 3 tane 1 biti içeriyorsa, sayı tek parity'ye sahiptir. Binary sayıda 1 biti yok ise, sayı çift parity'ye sahiptir.

PARITY FLAG (PF)

Bir işlemin sonucunda eğer 1 bitlerinin sayısı **çift** ise eşlik bayrağı aktif (1) olur. Eğer sonuçta bulunan 1 bitlerinin sayısı **tek** ise eşlik bayrağı pasif (0) olur. Sonuç 16 bitlik bile olsa sadece düşük değerli 8 bit ele alınır.

Örneğin;

```

01100100
10011000
+-----
11111100 → 1 bitlerinin sayısı 6 yani çift sayıda 1 biti var. Bu nedenle PF=1 (aktif)

01100100
00011000
+-----
01111100 → 1 bitlerinin sayısı 5 yani tek sayıda 1 biti var. Bu nedenle PF=0 (pasif)

```

- **A (Auxiliary Carry):** 3 ve 4. bit pozisyonları için geçerli olmak üzere, toplama işleminden oluşan elde ve çıkarma işleminden oluşan ödünçleri tutar.

AUXILIARY FLAG (AF)

İşaretsiz sayılarda yapılan işlemlerdeki düşük değerli 4 bitte taşma meydana gelirse AF **aktif** (1) olur. Eğer herhangi bir taşma meydana gelmezse AF **pasif** (0) olur. 4 bit ile 0-15 arasında sayılar ifade edilebilir.

Örneğin;

```

00001111 (15)
00000001 (1)
+-----
00010000 (16)

```

- **D (Direction):** DI ve SI register'ları için arttırma veya azaltma modlarından birini seçer. Özellikle büyük boyutlu veri transferinde verinin gidiş yönünü belirtir.

DIRECTION FLAG (DF)

Yön bayrağı diziler gibi ardışık verilerde özellikle string (metin) işlemlerinde kullanılan komutların ileri yönlü mü yoksa geri yönlü mü çalışacağını belirlemek için kullanılır. **DF=0** iken ileri yönlü işlem yapılır. **DF=1** iken geri yönlü işlem yapılır.

İleri Yönlü → Düşük Adresten Yüksek Adrese
Geri Yönlü → Yüksek Adresten Düşük Adrese

00000	T
00001	Ü
00002	R
00003	K
00004	İ
00005	Y
00006	E

İLERİ YÖNLÜ
GERİ YÖNLÜ

- **I (Interrupt):** INTR (interrupt request – kesme isteği) girişini kontrol eder.

INTERRUPT ENABLE FLAG (IF)

Bu bayrak **aktif** (1) durumda olduğu zaman işlemciye harici cihazlardan kesme sinyalleri gönderilebilir.

- **T (Trap):** Çip üzeri debug özelliğine olanak tanır.

- **Adresleme Modları:** Belleğin nasıl kullanıldığını, belleğe nasıl erişileceğini ve verilerin belleğe nasıl yerleştirileceğini belirler.

- **1) Anlık Adresleme (Immediate):** Doğrudan sabit bir değer, bir register'a aktarılır. Sabit değerın büyüklüğü ile register uyumlu olmalıdır. Örneğin 8 bitlik bir register alanına (AL, CH vb. gibi) 16 bitlik bir değer yüklenemez.

KOMUT register, değer

ADRESLEME MODLARI

Immediate

1-Veri Tanımlı Adresleme: Kaydedici içerisine programcı tarafından veri aktarılır.

MOV CL,18h ✓	MOV AL,'A' ✓
MOV CX,1814h ✓	MOV BX,'AB' ✓
MOV AL,4508h x	MOV BL, 1100b ✓
MOV AX,45h ✓ 0045h	
MOV AX,0FF45h ✓ ön ek olarak 0 eklenmelidir.	

- **2) Kaydedici Adresleme (Register Addressing):** Bu adresleme modunda her iki operand da register olarak bulunur.

KOMUT register, register

ADRESLEME MODLARI

Register

2-Kaydediciye Dayalı Adresleme: Kaydediciler kullanılarak veri alışverişi gerçekleştirilir.

MOV AX,BX ✓	
MOV AL,BX x	→ 8 bitlik kaydediciye 16 bitlik veri aktarılamaz.
MOV DS,ES x	→ Segment kaydedicileri birbirine aktarım yapamaz.
MOV CS,AX x	→ CS(Code Segment) içeriği değiştirilemez.
MOV DX,CS ✓	→ CS(Code Segment) içeriği başka bir kaydediciye aktarılabilir.
MOV SI,DI ✓	

- **3) Doğrudan Adresleme (Direct Addressing):** Doğrudan bir adres değeri kullanılır. Bir adresten bir kaydediciye veri aktarımı gerçekleştirilir. Bir başka ifade ile operandlardan birisi adres belirtir. Adresler için köşeli parantez kullanılır.

KOMUT register, [adres]

KOMUT [adres], register

ADRESLEME MODLARI

Direct

3-Doğrudan Adresleme: Adres bizzat programcı tarafından verilir.

MOV AL,[0104h] ✓	→ Adres 16 bitlik olacak ancak veri 8 bit alınacaktır.
MOV [0104h], AL ✓	
MOV [0104h], [0105h] x	

```
org 100h
MOV AL, sayi1
ret
sayi1 db 8
```

→ sayi1 aslında RAM üzerindeki bir byte boyutundaki bir hücreyi işaretleyen bir etikettir. MOV komutu ile 8 değeri AL içerisine aktarılacaktır.

MOV AX, [1000]

TOPLAM DW 20; Burada TOPLAM bir adrestir. 20 sayısı bu adresin içindeki değerdir.

MOV AX, TOPLAM; TOPLAM adresindeki değeri AX e at.

MOV TOPLAM, AX;

- 4) **Dolaylı Adresleme (Indirect Addressing):** Etkin adres değeri (offset) BX, BP, SI, DI kaydedicilerinden birinde bulunur. Burada BX register'ı BL ve BH olarak kullanılmaz.

KOMUT register, [BX/BP/SI/DI]

KOMUT [BX/BP/SI/DI], register

ADRESLEME MODLARI

Indirect

4-Kaydediciye Dayalı Dolaylı Adresleme: Adres bir kaydedicide tutulur.

```

org 100h
MOV BX,offset Dizi
MOV AL,[BX]
INC BX
INC BX
INC BX
MOV AH,[BX]
ret
Dizi DB 5,9,-1,2
  
```

Dizi olarak adlandırılan aslında bir RAM hücresidir. Bu RAM hücresinin adresini **offset** ile kaydediciye aktarabiliriz.

MOV [BP],CL → ✓
 MOV [BP],[BX] → ✗ bellek hücresi olamaz.
 MOV [BP],BH → ✓
 MOV [DI],[BX] → ✗ bellek hücresi olamaz.

MOV AX,[SI]

MOV BX,1000

SUB DX,[BX]

MOV [SI],AL

TABLO DB 5,9,0,3,-7

MOV BX,OFFSET TABLO; ¹Bunun yerine LEA BX,TABLO ²kullanılabilir.

; LEA komutu (Load Effective Address) offset adresini bir kaydediciye yüklemek için kullanılır.

Herhangi bir bellek bölgesi dolaylı bir adresleme ile adreslenip içine sabit bir değer atanmak istendiği zaman atanacak değerin uzunluğu **byte ptr** ve **word ptr** ile belirtilmelidir.

MOV [BX],12; yanlış

MOV byte ptr [BX],12; doğru 12 sabit değeri bayt olarak BX in gösterdiği adrese yerleşecek.

MOV word ptr [BX],1234; doğru 1234 sabit değeri word olarak BX in gösterdiği adrese yerleşecek.

- 5) **Taban Adresleme (Based Addressing):** Register (BX/BP) üzerinde tutulan adresin üzerine sayı eklenir.

KOMUT register, [BX/BP+1]

KOMUT [BX/BP+2], register

ADRESLEME MODLARI

Based

5-Kaydediciye Dayalı Göreceli: Kaydedicide tutulan adresin üzerine sayı eklenir.

```

org 100h
MOV BX,offset Dizi
MOV AL,[BX]
MOV AL,[BX]+1
MOV AL,[BX+2]
ret
Dizi DB 5,9,-1,2
  
```

Dizinin ilk elemanı olan 5 değeri AL kaydedicisine aktarılır.
 Dizinin ikinci elemanı olan 9 değeri AL kaydedicisine aktarılır
 Dizinin üçüncü elemanı olan -1 değeri AL kaydedicisine aktarılır

[BX+2] ifadesi ile [BX]+2 ifadesi aynıdır.

NOT: Atama işlemlerinde değişken/dizi boyutunun ne olduğuna dikkat edin.
 DB ile tanımlı bir değişken/dizi değerini **8 bitlik** kaydedicilere(AH,AL,BH,BL,CH,CL,DH,DL) atanması gerekir. DW ile tanımlı bir değişken/dizi değerini **16 bitlik** kaydedicilere (AX,BX,CX,DX) atanması gerekir.

- 6) **Indis Adresleme (Indexed Addressing)**: Register üzerinde tutulan adresin üzerine indis register'larından biri eklenir.

KOMUT **register**, [BX/BP+SI/DI]

KOMUT [BX/BP+SI/DI], **register**

ADRESLEME MODLARI

Indexed

6-Taban+İndeks Adresleme: Kaydedicide bulunan taban adres ile indeks kaydedicileri kullanılır.

Başlangıç (Taban): BX, BP
İndeks (Offset): DI, SI

```

org 100h

mov bx,offset dizi
mov di,0001
mov al,[bx+di] ;9 değerini AL'ye aktarır
inc di
mov [bx+di],al ;dizinin 3. elemanı 9 olur

ret
Dizi DB 5,9,-1,2

```

- 7) **Taban ve İndis Adresleme (Based-Indexed Addressing)**: Register üzerinde tutulan adresin üzerine sayı ve indis register'larından biri eklenir.

KOMUT **register**, [BX/BP+SI/DI+1]

KOMUT [BX/BP+SI/DI+2], **register**

ADRESLEME MODLARI

Based Indexed

7-Taban Göreceli+İndis Adresleme: Kaydedicide bulunan taban adrese bir yayılım(sayı) değeri eklenir. İndeks kaydedicilerindeki değer ile toplanır ve bir RAM hücreğine erişim sağlanır.

Başlangıç (Taban): BX, BP
İndeks (Offset): DI, SI

```

org 100h

mov bx,offset dizi
mov di,0001
mov al,[bx+1+di] ;-1 değerini AL'ye aktarır
inc di
mov [bx+1+di],al ;dizinin 4. elemanı -1 olur

ret
Dizi DB 5,9,-1,2

```

• **Veri aktarımında dikkat edilmesi gerekenler:**

- KOMUT [adres], [adres] şeklinde bir veri aktarımı yapılamaz!
- Sabit bir değer, doğrudan Segment Registerine atanamaz!

MOV DS, 1234; **Yanlıştır**. Bunun yerine aşağıdaki kullanım geçerlidir.

MOV AX, 1234; Önce kaydediciye,

MOV DS, AX; Sonra segment kaydedicisine değer atanır.

16 Bitlik bir verinin belleğe yerleşmesi

Belleğin her bir hücresi 8 bit olduğundan 16 bitlik verinin düşük değerlikli 8 bitlik kısmı adresin düşük değerlikli kısmına, yüksek değerlikli 8 bitlik kısmı adresin yüksek değerlikli kısmına yerleşir. Aşağıda verilen örnekte AX kaydedicisinde 2A8BH değeri yer almaktadır. Düşük değerlikli değer AL de yer almakta ve 10H adresine yerleşmektedir. Yüksek değerlikli kısımda yer alan AH daki değer 2AH değeri ise 11H adresine yerleşmektedir.

- MOV AX, 2A8BH

- MOV [10H], AX

08H	??
09H	??
10H	8B
11H	2A
12H	??
13H	??

AX: 2A 8B

BX: 83 F4

BH: BL

Benzer olarak 32bitlik bir veriyi(doubleword) bu şekilde düşünerek yerleştirebilirsiniz.

Örneğin:

SAYI DB 1A2F56FFH verisini 100H adresinden itibaren yerleştirecek olursa şu şekilde olur.

Adres	veri
100	FF
101	56
102	2F
103	1A
104	
105	
106	

- **Dizi Tanımlaması ve Elemanlarına Erişimi:**

DIZI DB 5, 6, 7, 8, -6, -9, 3; *şeklinde tanımlaması yapılır.*

LEA SI, DIZI; *şeklinde dizinin başlangıç adresi SI kaydedicisine alınır.*

veya

MOV BX, OFFSET DIZI; *şeklinde dizinin başlangıç adresi BX kaydedicisine alınır.*

- **ADC Komutu:** Elde'li toplama işlemini gerçekleştirir. Elde'li toplama olduğu için, ilgili toplama CF (Carry Flag) bitindeki 1'i de ekler.

ADC	REG, memory memory, REG REG, REG memory, immediate REG, immediate	Add with Carry. Algorithm: $\text{operand1} = \text{operand1} + \text{operand2} + \text{CF}$ Example: <pre>STC ; set CF = 1 MOV AL, 5 ; AL = 5 ADC AL, 1 ; AL = 7 RET</pre> <table border="1"><tr><td>C</td><td>Z</td><td>S</td><td>O</td><td>P</td><td>A</td></tr><tr><td>r</td><td>r</td><td>r</td><td>r</td><td>r</td><td>r</td></tr></table>	C	Z	S	O	P	A	r	r	r	r	r	r
C	Z	S	O	P	A									
r	r	r	r	r	r									

- **ADD Komutu:** Elde'siz toplama işlemini gerçekleştirir.

ADD	REG, memory memory, REG REG, REG memory, immediate REG, immediate	Add. Algorithm: $\text{operand1} = \text{operand1} + \text{operand2}$ Example: MOV AL, 5 ; AL = 5 ADD AL, -3 ; AL = 2 RET <table border="1"><tr><td>C</td><td>Z</td><td>S</td><td>O</td><td>P</td><td>A</td></tr><tr><td>r</td><td>r</td><td>r</td><td>r</td><td>r</td><td>r</td></tr></table>	C	Z	S	O	P	A	r	r	r	r	r	r
C	Z	S	O	P	A									
r	r	r	r	r	r									

- **SBB Komutu:** Elde'li çıkarma işlemini gerçekleştirir. Elde olduğu için, ilgili işlemde CF biti de çıkarılır.

SBB

REG, memory
memory, REG
REG, REG
memory, immediate
REG, immediate

Subtract with Borrow.

Algorithm:

$\text{operand1} = \text{operand1} - \text{operand2} - \text{CF}$

Example:

```
STC  
MOV AL, 5  
SBB AL, 3 ; AL = 5 - 3 - 1 = 1
```

RET

C	Z	S	O	P	A
r	r	r	r	r	r

- **SUB Komutu:** Elde'siz çıkarma işlemini gerçekleştirir.

SUB	REG, memory memory, REG REG, REG memory, immediate REG, immediate	Subtract. Algorithm: $\text{operand1} = \text{operand1} - \text{operand2}$ Example: <pre>MOV AL, 5 SUB AL, 1 ; AL = 4 RET</pre> <table border="1"><tr><td>C</td><td>Z</td><td>S</td><td>O</td><td>P</td><td>A</td></tr><tr><td>r</td><td>r</td><td>r</td><td>r</td><td>r</td><td>r</td></tr></table>	C	Z	S	O	P	A	r	r	r	r	r	r
C	Z	S	O	P	A									
r	r	r	r	r	r									

- **MUL Komutu:** İşaretsiz sayılarda **çarpma işlemi** yapar.

MUL

REG
memory

Unsigned multiply.

Algorithm:

when operand is a **byte**:
 $AX = AL * \text{operand.}$

when operand is a **word**:
 $(DX AX) = AX * \text{operand.}$

Example:

```
MOV AL, 200 ; AL = 0C8h  
MOV BL, 4  
MUL BL ; AX = 0320h ($00)  
RET
```

C	Z	S	O	P	A
r	?	?	r	?	?

CF=OF=0 when high section of the result is zero.

- **IMUL Komutu:** İşaretli sayılarda çarpma işlemi yapar.

IMUL REG memory	Signed multiply. Algorithm: when operand is a byte: $AX = AL * \text{operand}$. when operand is a word: $(DX AX) = AX * \text{operand}$. Example: <pre>MOV AL, -2 MOV BL, -4 IMUL BL ; AX = 8 RET</pre>	 CF=OF=0 when result fits into operand of IMUL
-----------------------------------	---	--

- **DIV Komutu:** İşaretsiz sayılarda bölme işlemi yapar.

DIV

REG
memory

Unsigned divide.

Algorithm:

when operand is a **byte**:
 AL = AX / operand
 AH = remainder (modulus)

when operand is a **word**:
 AX = (DX AX) / operand
 DX = remainder (modulus)

Example:

```
MOV AX, 203 ; AX = 00CBh
MOV BL, 4
DIV BL ; AL = 50 (32h), AH = 3
RET
```

C	Z	S	O	P	A
?	?	?	?	?	?

8bit

AX	op
	AL
AH	

16bit

DX.AX	op
	AL
DX	

- **IDIV Komutu:** İşaretili sayılarda bölme işlemi yapar.

IDIV	REG memory	Signed divide.
		Algorithm: when operand is a byte: $AL = AX / \text{operand}$ $AH = \text{remainder (modulus)}$ when operand is a word: $AX = (DX AX) / \text{operand}$ $DX = \text{remainder (modulus)}$ Example: MOV AX, -203 ; AX = 0FF35h MOV BL, 4 IDIV BL ; AL = -50 (0CEh), AH = -3 (0FDh) RET

- **STC Komutu:** CF bayrağını 1 yapar. ADC komutu ile birlikte kullanılabilir.
- **CLC Komutu:** CF bayrağını 0 yapar.
- **CMP Komutu:** Karşılaştırma işlemi yapar.

CMP	REG, memory memory, REG REG, REG memory, immediate REG, immediate	Compare. Algorithm: $operand1 - operand2$ result is not stored anywhere, flags are set (OF, SF, ZF, AF, PF, CF) according to result. Example: <pre>MOV AL, 5 MOV BL, 5 CMP AL, BL ; AL = 5, ZF = 1 (so equal!) RET</pre> CZSOPA r r r r r r
-----	---	---

- **LOOP Komutu:** Döngü oluşturmak için kullanılır. CX kaydedicisi ile birlikte çalışır. Her döngü adımında otomatik olarak CX'teki değer 1 azalır. CX değeri sıfır olduğunda döngü sonlanır.

LOOP	label	Decrease CX, jump to label if CX not zero. Algorithm: • $CX = CX - 1$ • if $CX < 0$ then ◦ jump - else ◦ no jump, continue Example: <pre>include 'emu8086.inc' ORG 100h MOV CX, 5 label1: PRINTN 'loop!' LOOP label1 RET</pre> CZSOPA unchanged
------	-------	--

- **JMP Komutu:** Koşulsuz dallanma komutudur. Herhangi bir koşula bağlı olmadan, belirtilen etiket satırına gider.

JMP	label 4-byte address	Unconditional Jump. Transfers control to another part of the program. 4-byte address may be entered in this form: 1234h:5678h, first value is a segment second value is an offset. Algorithm: always jump Example: <pre>include 'emu8086.inc' ORG 100h MOV AL, 5 JMP label1 ; jump over 2 lines! PRINT 'Not Jumped!' MOV AL, 0 label1: PRINT 'Got Here!' RET</pre> CZSOPA unchanged
-----	-------------------------	---

- **JA Komutu:** CMP komutundan sonra ve **işaretsiz sayılar için** kullanılır. **operand1 > operand2** ise belirtilen etikete gider.

JA	label	Short Jump if first operand is Above second operand (as set by CMP instruction). Unsigned. Algorithm: if (CF = 0) and (ZF = 0) then jump Example: <pre>include 'emu8086.inc' ORG 100h MOV AL, 250 CMP AL, 5 ; AL değeri 5'ten büyükse JA komutuyla label1: etiketine atlanır (dallanır). JA label1 PRINT 'AL is not above 5' JMP exit label1: PRINT 'AL is above 5' exit: RET</pre> <table border="1"> <tr> <td>C</td><td>Z</td><td>S</td><td>O</td><td>P</td><td>A</td> </tr> <tr> <td colspan="6">unchanged</td> </tr> </table>	C	Z	S	O	P	A	unchanged					
C	Z	S	O	P	A									
unchanged														

- **JB Komutu:** CMP komutundan sonra ve **işaretsiz sayılar için** kullanılır. **operand1 < operand2** ise belirtilen etikete gider.

JB	label	Short Jump if first operand is Below second operand (as set by CMP instruction). Unsigned. Algorithm: if CF = 1 then jump Example: <pre>include 'emu8086.inc' ORG 100h MOV AL, 1 CMP AL, 5 ; AL değeri 5'ten küçükse JB komutuyla dallanır. (JA tersi) JB label1 PRINT 'AL is not below 5' JMP exit label1: PRINT 'AL is below 5' exit: RET</pre> <table border="1"> <tr> <td>C</td><td>Z</td><td>S</td><td>O</td><td>P</td><td>A</td> </tr> <tr> <td colspan="6">unchanged</td> </tr> </table>	C	Z	S	O	P	A	unchanged					
C	Z	S	O	P	A									
unchanged														

- **JAE Komutu:** CMP komutundan sonra ve **işaretsiz sayılar için** kullanılır. **operand1 >= operand2** ise belirtilen etikete gider.

JAE	label	Short Jump if first operand is Above or Equal to second operand (as set by CMP instruction). Unsigned. Algorithm: if CF = 0 then jump Example: <pre>include 'emu8086.inc' ORG 100h MOV AL, 5 CMP AL, 5 JAE label1 PRINT 'AL is not above or equal to 5' JMP exit label1: PRINT 'AL is above or equal to 5' exit: RET</pre> AL değeri büyük veya eşitse JAE etiketine dallanır. <table border="1"> <tr> <td>C</td><td>Z</td><td>S</td><td>O</td><td>P</td><td>A</td> </tr> <tr> <td colspan="6">unchanged</td> </tr> </table>	C	Z	S	O	P	A	unchanged					
C	Z	S	O	P	A									
unchanged														

- **JBE Komutu:** CMP komutundan sonra ve **işaretsiz sayılar için** kullanılır. **operand1 <= operand2** ise belirtilen etikete gider.

JBE	label	Short Jump if first operand is Below or Equal to second operand (as set by CMP instruction). Unsigned. Algorithm: if CF = 1 or ZF = 1 then jump Example: <pre>include 'emu8086.inc' ORG 100h MOV AL, 5 CMP AL, 5 JBE label1 PRINT 'AL is not below or equal to 5' JMP exit label1: PRINT 'AL is below or equal to 5' exit: RET</pre> AL değeri küçükse veya eşitse JBE etiketine dallanır. CZSOPA unchanged
-----	-------	---

- **JE Komutu:** CMP komutundan sonra ve **işaretili/işaretsiz** sayılar için kullanılır. **operand1 = operand2** ise belirtilen etikete gider.

JE	label	Short Jump if first operand is Equal to second operand (as set by CMP instruction). Signed/Unsigned. Algorithm: if ZF = 1 then jump Example: <pre>include 'emu8086.inc' ORG 100h MOV AL, 5 CMP AL, 5 JE label1 PRINT 'AL is not equal to 5.' JMP exit label1: PRINT 'AL is equal to 5.' exit: RET</pre> CZSOPA unchanged
----	-------	---

- **JZ Komutu:** CMP, SUB, ADD, TEST, AND, OR, XOR komutlarından sonra ve **işaretili/işaretsiz** sayılar için kullanılır. **operand1 = operand2** ise belirtilen etikete gider. Zero bayrağı 1 olduğu zaman çalışır. Bir **işlemin sonucu SIFIR** çıkmış demektir.

JZ	label	Short Jump if Zero (equal). Set by CMP, SUB, ADD, TEST, AND, OR, XOR instruction. Algorithm: if ZF = 1 then jump Example: <pre>include 'emu8086.inc' ORG 100h MOV AL, 5 CMP AL, 5 JZ label1 PRINT 'AL is not equal to 5.' JMP exit label1: PRINT 'AL is equal to 5.' exit: RET</pre> CZSOPA unchanged
----	-------	---

- **JNE/JNZ Komutu:** JE ve JZ komutlarının tersidir. **operand1 != operand2** ise belirtilen etikete gider. (ZF=0 ise)

- **JC Komutu:** Bir işlemin sonucunda **CF=1** oluyorsa, belirtilen etikete gider.

JC	label	Short Jump if Carry flag is set to 1. Algorithm: if CF = 1 then jump Example: <pre>include 'emu8086.inc' ORG 100h MOV AL, 255 ADD AL, 1 JC label1 PRINT 'no carry.' JMP exit label1: PRINT 'has carry.' exit: RET</pre> CZSOPA unchanged
----	-------	---

- **JNC Komutu:** Bir işlemin sonucunda **CF=0** oluyorsa, belirtilen etikete gider.
- **JG Komutu:** CMP komutundan sonra ve **işaretili sayılar** için kullanılır. **operand1 > operand2** ise belirtilen etikete gider.

JG	label	Short Jump if first operand is Greater then second operand (as set by CMP instruction). <u>Signed</u>. Algorithm: if (ZF = 0) and (SF = OF) then jump Example: <pre>include 'emu8086.inc' ORG 100h MOV AL, 5 CMP AL, -5 JG label1 PRINT 'AL is not greater -5.' JMP exit label1: PRINT 'AL is greater -5.' exit: RET</pre> CZSOPA unchanged
----	-------	--

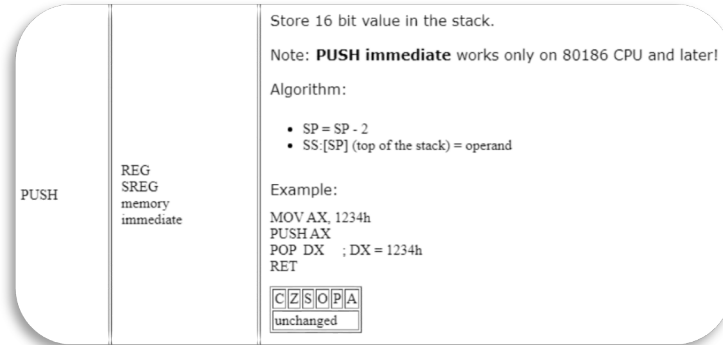
- **JL Komutu:** CMP komutundan sonra ve **işaretili sayılar** için kullanılır. **operand1 < operand2** ise belirtilen etikete gider.
- **CALL Komutu:** Önceden oluşturulan prosedürleri (bir nevi fonksiyon) çalıştırmak için kullanılır. İçinde mutlaka **RET** komutu olmalıdır.

CALL	procedure name label 4-byte address	Transfers control to procedure. Return address (IP) is pushed to stack. 4-byte address may be entered in this form: 1234h:5678h, first value is a segment second value is an offset. If it's a far call, then code segment is pushed to stack as well. Example: <pre>ORG 100h ; for COM file. CALL p1 ADD AX, 1 RET ; return to OS. p1 PROC ; procedure declaration. MOV AX, 1234h RET ; return to caller. p1 ENDP</pre> CZSOPA unchanged <pre>ORG 100h call fonk add ax, 1 ret fonk proc mov ax, 1234h ret fonk endp</pre>
------	---	--

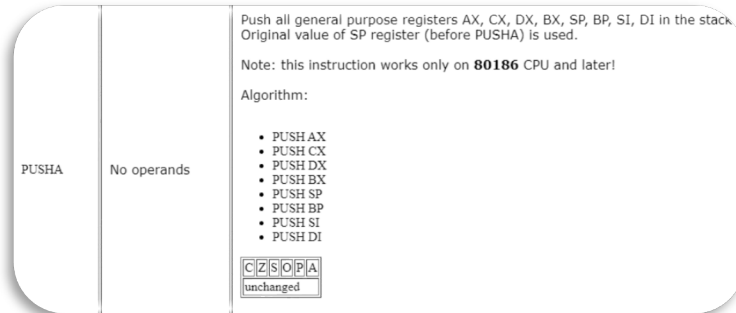
- **AND/OR/XOR Komutları:** Sırasıyla VE/VEYA/XOR **mantıksal işlemleri** yapmak için kullanılırlar.

AND	REG, memory memory, REG REG, REG memory, immediate REG, immediate	Logical AND between all bits of two operands. Result is stored in operand1. These rules apply: <pre>1 AND 1 = 1 1 AND 0 = 0 0 AND 1 = 0 0 AND 0 = 0</pre> Example: <pre>MOV AL, 'a' ; AL = 01100001b AND AL, 11011111b ; AL = 01000001b ('A') RET</pre> CZSOPA 01000001
-----	---	---

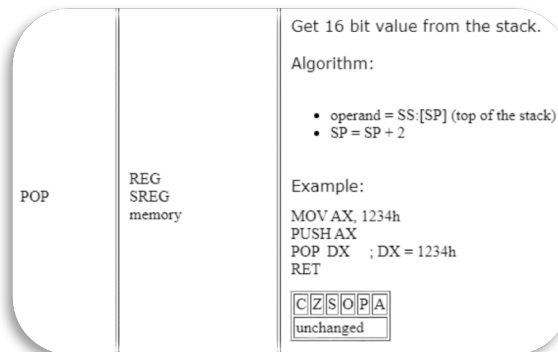
- **NOT Komutu:** 0 bitlerini 1 ve 1 bitlerini 0 yapar.
- **INC Komutu:** İlgili operand'ı 1 arttırır. Burada operand register veya [adres] (yani memory) olmalıdır. $\text{Operand} = \text{Operand} + 1$
- **DEC Komutu:** İlgili operand'ı 1 azaltır. Burada operand register veya [adres] (yani memory) olmalıdır. $\text{Operand} = \text{Operand} - 1$
- **ROL Komutu:** Bitleri sola kaydırırken, silinen bitleri en sağa ekler. Bitleri sola kaydırmak, 2 ile çarpmak demektir. (Rotate)
- **ROR Komutu:** Bitleri sağa kaydırırken, silinen bitleri en sola ekler. Bitleri sağa kaydırmak, 2'ye bölmek demektir. (Rotate)
- **SHL Komutu:** Bitleri sola kaydırırken, silinen her bitin yerine en sağa 0 ekler.
- **SHR Komutu:** Bitleri sağa kaydırırken, silinen her bitin yerine en sola 0 ekler.
- **PUSH Komutu:** Stack belleğe veri ekler. **SP (Stack Pointer) = SP - 2 olarak setlenir.** Tüm push komutları 16 bitlik işlem yapar.



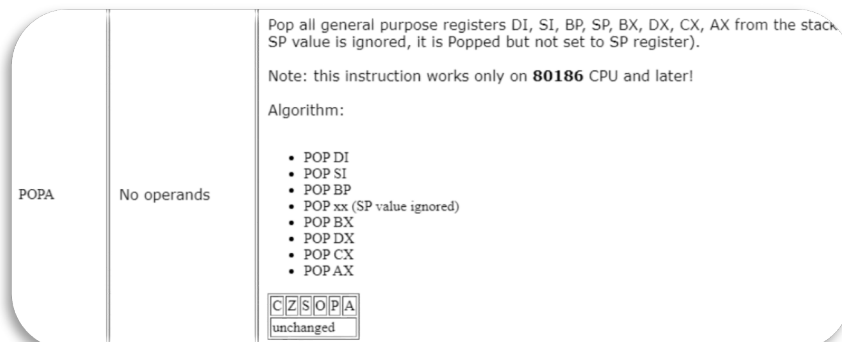
- **PUSHA Komutu:** Tüm özel register değerlerini (AX,BX,CX,DX,SP,BP,SI,DI) stack belleğe ekler.



- **PUSHF Komutu:** Flag register değerlerini stack belleğe atar.
- **POP Komutu:** Stack bellekten veri alır. **SP = SP + 2 olarak setlenir.**



- **POPA Komutu:** Tüm özel register değerlerini (AX,BX,CX,DX,SP,BP,SI,DI) stack bellekten çıkartır.



- **POPF Komutu:** Flag register değerlerini stack bellekten çıkartır.

- **Prosedürler:** Alt programlardır. **CALL** komutu ile çalıştırılırlar. Yazılan prosedür **aynı segmentte** ise buna **yakın dallanma** veya **yakın alt program** denir. **Farklı segment adresinde** ise **uzak dallanma** veya **uzak alt program** denir. Uzak dallanmada, **CALL FAR Fonksiyon** şeklinde FAR kelimesini de komuta eklemek gerekir.
- **Stack Bellek:** Stack bellek bütün işlemcilerde olan bir yapıdır. Olmazsa olmazdır. Fonksiyonun (prosedürün) işi bittiğinde, **daha önce nerde kaldığını bilmesi için** stack bellekten yararlanılır. Normal ramlerden daha hızlı olacak şekilde tasarlanırlar. Intel 8086'da stack bellek kapasitesi **65536 byte**'tir. **Yakın alt program için sadece offset adresi** tutulur. Segment adresinin tutulmasına gerek yoktur. **Uzak alt programda** ise **programın daha önce kaldığı segment adresinin de** tutulması gerekir. Yakın alt programın kaldığı yer stack'te 2 byte yer kaplarken, uzak alt programın kaldığı yer, stack'te 4 byte yer kaplar.
- **Prosedür çalıştırıldığında;**
 - **CS:IP** konumu (yani ana kodda, CALL komutunun bulunduğu konum) Stack belleğe kaydedilir.
 - Alt programın (prosedürün) başlangıç adresi, **CS:IP**'ye yüklenir. Alt program yürütülür.
 - RET komutuyla alt program sonlanınca, Stack bellekte saklanan **CS:IP** değeri geri yüklenir.
 - **CS:IP ← CS:IP + 1** işlemi ile daha önce kalınan yerden bir sonraki koda geçilir, ana kod yürütülmeye devam edilir.
- **CALL komutu algılandığında,** Stack bellekle neler olur?
 - **SS:SP ← CS:IP** (**CS:IP** Konumu Stack belleğe kaydedilir)
 - Yakın dallanmada; **SS:SP ← SS:SP - 2**
 - Uzak dallanmada; **SS:SP ← SS:SP - 4** olur.
- **RET komutu algılandığında,** Stack bellekte neler olur?
 - Yakın dallanmada; **SS:SP ← SS:SP + 2**
 - Uzak dallanmada; **SS:SP ← SS:SP + 4** olur.
 - **CS:IP ← SS:SP**
- **Interrupt (Kesmeler):** Kesme, bir işlemcinin **yaptığı bir işi bırakıp, yeni talep edilen işe yönelmesidir**. Interrupt geldiğinde, ana programımız interrupt alt programına dallanır. **IRET komutunu görene kadar alt programın çalışması devam eder**. IRET sonrası, ana program kaldığı yerden işlemeye devam eder.
- **Kesmeler,** **Yazılımsal ve Donanımsal** kesmeler olarak ikiye ayrılır. **Donanımsal kesmeler,** **maskelenebilir ve maskelenemez** kesmeler olarak ikiye ayrılır. **Maskelenebilir kesmeler:** **Timer interrupt, Ses kartı interruptı ve benzerleridir**. **Maskelenemez kesmeler:** **Ram failure, Power failure** gibi kesmelerdir.

KESME NEDİR?

Kesmeler işlemcinin daha önemli başka bir görevi yerine getirmesi için normal akışından çıkarak icra edilecek görevi yerine getirmesi için kullanılan mekanizmalardır.

Bu görevler tuş takımından bir karakter okunması, ekrana bir karakter yazılması, farenin konumunun saptanması gibi işlemler olabilir.

İki tür kesme bulunmaktadır. Birincisi, donanım kesmeleridir. İkincisi, yazılım kesmeleridir.

Donanım kesmelerinde işlemcinin kesme bacağına elektrik sinyali gönderilir ve olağan akışından çıkması sağlanabilir. Örneğin uyku moduna geçme gibi bir işlemde donanım kesmeleri kullanılabilir.

Yazılım kesmelerinde komut kümesi ve kesme tablosunda (interrupt vector table) bulunan önceden belirlenmiş değerler kullanılarak işlemcinin normal akışından çıkıp istenileni gerçekleştirmesi sağlanabilir. Burada istenilen klavyeden bir değer okumakta olabilir, ekrana bir yazı yazmakta olabilir.

YAZILIM KESMESİ TÜRLERİ

DOS: MS-DOS işletim sistemi temelli programlar tarafından kullanılan kesmelerdir.

BIOS: Herhangi bir işletim sistemine ihtiyaç duymayan direkt donanım üzerinde gerçekleştirilen fonksiyonları içerisinde barındıran kesmelerdir.

DOS Kesmeleri
INT 21h

BIOS Kesmeleri
INT 10h
INT 11h
...
...

- **Interrupt Vektör Adresi/Tablosu:** Interrupt'ın gelme anını bilemeyiz. Örneğin; kapının ne zaman çalınacağını, kişinin klavyeye ne zaman basacağını bilemeyeceğimiz gibi. Bu yüzden, Interrupt vektör adresi kullanılır. Bu adresler Interrupt vektör tablosunda bulunur. İşlemcinin desteklediği tüm interruptları tutan bir tablodur. Kullanıcıdan klavye yoluyla bilgi alınmak istendiğinde, vektör tablosunda bu işi yapacak olan ilgili interrupt tetiklenir ve kullanıcıdan veri alma işlemi gerçekleştirilmiş olur.
- **Interrupt algılandığında,** Intel 8086 işlemcisinde hangi işlemler gerçekleşir?
 - **Interrupt Bayrağı, sıfır olarak setlenir. (IF=0)** İşlemci aynı anda birden fazla interrupt'ı işleyemeyeceği için IF=0 yapılır. (Disable)
 - **Program Sayacının son kaldığı yer,** (PC - interrupt öncesi son kalınan yer) **Stack belleğe yazılır. (Stack Bellek ← CS:IP)**
 - **Program Status Word** (yani **Bayrak Register'i**), **Stack belleğe yüklenir**. Çünkü bayraklar, interruptta gerçekleşecek işlemde etkilenebilirler. Bunu önlemek için önceki halleri kaydedilmelidir.
 - Interrupt vektör adresine gidilir ve oradan, **Interrupt alt program başlangıç adresi** (yani **ISR - Interrupt Sub Routine**), **Program Sayacına yüklenir**.
 - **IRET komutuna kadar, ISR işletilir**. Interrupt'ın işi biter.
 - **Stack'e yazılan Bayrak Register'i, Program Status Word'e geri yüklenir**. Yani bayrakların interrupt öncesi hallerine geri dönülür.
 - Yine **Stack'e yazılan program sayacının son kaldığı yer, program sayacına yüklenir. (CS:IP ← Stack Bellek)**
 - **Interrupt Bayrağı, bir olarak setlenir. (IF=1)** Bu andan sonra, işlemci yeni interrupt'ları kabul edebilir. (Enable)
 - **PC ← PC+1** (Program Sayacı bir artırılır. Interrupt'tan sonraki komutların işletilmesi için)

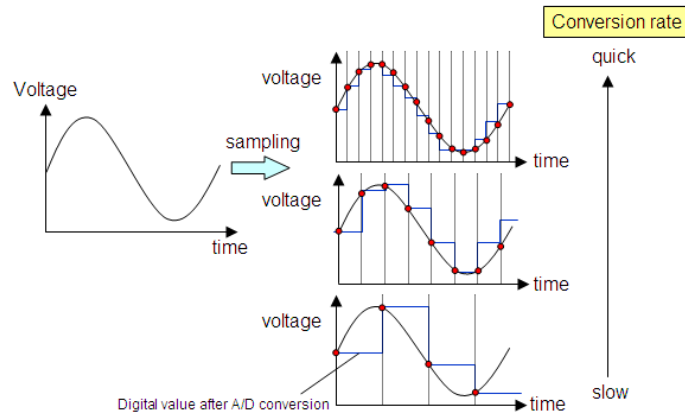
- **Polling:** Interrupt'ın kaynağını gösterir ve interrupt vektör adresinde bulunur.
- **STI Komutu:** IF bayrağını 1 olarak setler.
- **CLI Komutu:** IF bayrağını 0 olarak setler.
- **Sinyaller:** Sürekli zamanlı ve Ayrık zamanlı olarak ikiye ayrılır. Analog sinyaller, sürekli zamanlı; Dijital sinyaller ise, ayrık zamanlıdır.
- **Analog-Dijital Dönüşüm İşlemi:** Analog sinyallerin, örnekleme (sampling) yoluyla, dijitale (sayısal) çevrilmesi ve alınan her örneğin ölçeklendirilip, bitlere atanması (quantization) işlemidir.
- Analog bir sinyalin en doğru şekilde dijital bir sinyale çevrilebilmesi için;
 - Sinyalin bant genişliği belirlenmeli, yani frekans spektrumu (tayfı) olabilecek max bir frekans aralığında sınırlanmalı.
 - Örnekleme frekansına f_s dersek, $f_s \geq 2f_{max}$ esitliği sağlanmalı.
- **Nyquist Değeri:** Minimum örnekleme frekansıdır. $f_s = 2f_{max}$
- **Nyquist Frekansı:** Herhangi bir f_s değerinde $f_s/2$ ye karşılık gelen değerdir.

Örnek:

Frekansı 1 kHz olan bir sinyalin, doğru bir şekilde sayısal çevrilebilmesi için, minimum hangi örnekleme frekansı kullanılmalıdır?

$$f_s = 2 f_{max} = 2 * 1 \text{ kHz} = 2 \text{ kHz}$$

- **ADC:** Analog bir sinyali, (Ses sinyali, Işık sinyali vs.) sayısal (dijital) çeviren donanımların genel adıdır. ADC, analog sinyali alıp binary sayıya çevirir. Dolayısıyla ADC çıkışındaki binary sayı, analog voltaj seviyesini de ifade eder. **Çözünürlük**, bir ADC'nin yakalayabileceği (ölçebileceği) en küçük voltaj değişimidir.



- **ADC Çözünürlüğü:** Çözünürlük; n bitlik bir ADC'nin, analog sinyalin maksimum kaç parçaya ayrılacağını ifade etmekte kullanılan terimdir.
- **Çözünürlük hesaplama:** bitlik bir ADC'nin çözünürlüğü = 2^n dir. 10 bitlik bir ADC, $2^{10} = 1024$ çözünürlüğe sahiptir. Yani en iyi çözünürlük değerimiz 1/1024 veya tam skalanın, %0.0976'sıdır.
- **Çözünürlük, ölçümün hassasiyetinin sınırlarını belirler.** Yüksek çözünürlük, daha hassas ölçüm manasına gelir. 8 bitlik bir ADC ile 10V genlik değerine sahip bir sinyali örneklersek, teorik olarak 39mV'luk genlik değişimini algılayabilecekken, aynı sinyali 14 bitlik ADC ile sayısal veriye dönüştürdüğümüzde 610µV luk değişimleri ölçebiliriz. Bit sayısı arttıkça, daha küçük değişimleri de ölçebilmiş oluruz.

Örnek:

10 bitlik bir ADC girişine, 3.25V uygulanmaktadır. ADC çıkışındaki değeri hesaplayınız. ($V_{ref+} = 5V$, $V_{ref-} = GND$)

$$[(V_{ref+}) - (V_{ref-})] / 2^{10} = (5 - 0) / 1024 = 0.004882 \text{ V} = 4.882 \text{ mV (Giriş Ref Volt / Çözünürlük)}$$

$$3.25 \text{ V} = 3250 \text{ mV olur.}$$

$$\text{ADC Çıkışı} = 3250 / 4.882 = 665.710 \sim (1010011001)_2$$

- **Mikroişlemcilerde analog sinyal çıkışı,** **DAC (Sayısal Analog Çevirici)** ve **PWM (Darbe Genliği Modülasyonu)** ile sağlanır.
- **DAC:** ADC'ler ile sayısal olarak kaydedilen veriler, DAC devreleri yardımıyla tekrardan analog sinyale çevrilebilirler. Dijital-analog dönüştürücüler, ikili sayıları analog eşdeğer voltajlarına dönüştürür.
- **PWM:** Sayısal sinyalin darbe genişliğinin değiştirilerek, yüke veya bir devre elemanına uygulanan gücün kontrol edilmesini sağlayan modülasyon tekniğidir.
- **PWM Çözünürlüğü:** 1 PWM periyoduna, maksimum kaç farklı darbeyi (duty cycle sayısını) tanımlayabileceğimizi ifade eden terimdir.
- **PWM Uygulamaları:**
 - **Aydınlatma uygulamaları**
 - **DC Motor ve Fan hız kontrol uygulamaları**
 - **Ayarlanabilir voltaj kaynaklarında**
 - **Servo motorların yön ve derece kontrolünde**
 - **Analog sinyal üretme**

- **PIO 8255:** Paralel giriş-çıkış bağlantı noktası yongası 8255, programlanabilir çevresel giriş-çıkış bağlantı noktası olarak da adlandırılır. Portlarda veri alışverişini sağlar. Intel 8255, Intel'in 8 bit, 16 bit ve daha yüksek kapasiteli mikroişlemcileriyle kullanılmak üzere tasarlanmıştır. Her biri oniki satırlık iki grupta veya sekiz satırlık üç grupta ayrı ayrı programlanabilen 24 giriş / çıkış hattına sahiptir.

- **8255 I/O Modları:** Porttan veri almak için giriş/çıkış'ını bilmek gerekir. İşlemci direkt porttan veri okumaz, 8255 gibi arabirimler kullanır.
 - **Mod 0 (Temel Giriş / Çıkış modu):** Bu mod, portların her birini kullanarak basit giriş ve çıkış özellikleri sağlar. Veriler, uygun başlatma işleminden sonra giriş ve çıkış portlarından sırasıyla kolayca okunabilir ve bunlara yazılabilir.
 - **Mod 1 (Strobed Giriş / Çıkış modu):** Bu modda el sıkışma, belirtilen portun giriş ve çıkış eylemlerini kontrol eder. Port C hatları, PC0-PC2; Port B için el sıkışma hatları sağlar. Port B ve PC0-PC2'yi içeren bu gruba Strobed veri girişi / çıkışı için grup B adı verilir. Port C hatları PC3-PC5, A portu için strobe hatları sağlar. A grubu A portu ve PC3-PC5'i içerir.
 - **Mod 2 (Strobed çift yönlü Giriş / Çıkış):** 8255'in bu çalışma moduna strobe çift yönlü Giriş / Çıkış denir. Bu çalışma modu, 8255'e 8 bitlik bir veri yolunda bir çevrebirim cihazı ile iletişim kurmak için ek özellikler sağlar. Veri aktarıcı ve alıcı arasında doğru veri akışını ve senkronizasyonu sağlamak için el sıkışma sinyalleri sağlanır.
 - **Bit Set Reset Modu (BSR):** Bu modda, C portunun 8 bitinden herhangi biri, kontrol kaydedicisinin (Control Word) D0 değerine bağlı olarak ayarlanabilir veya sıfırlanabilir. Ayarlanacak veya sıfırlanacak bit, tabloda belirtildiği gibi Kontrol Kaydedicisinin D3, D2 ve D1 bit seçim bayrakları ile seçilir.
- **Paralel Port (Paralel İletişim):** Sekiz ayrı kablo veya hat üzerinden bir seferde sekiz bit veri gönderir ve alır. Bu, verilerin çok hızlı bir şekilde aktarılmasını sağlar. Bununla birlikte, kurulum içermesi gereken kabloların sayısı nedeniyle daha hantal görünür. Ancak, seri iletişim söz konusu olduğunda, bir seri port bir kablo üzerinden birer birer veri gönderir ve alır. Her bir bayt veriyi bu şekilde aktarmak sekiz kat daha uzun sürse de, sadece birkaç kablo gereklidir.
- **Seri İletişim,** paralel iletişimden daha yavaş olmasına rağmen, daha basittir ve daha uzun mesafelerde kullanılabilir.
- **Seri İletişimin paralel iletişime göre avantajlı olduğu noktalar:**
 - Cross talk "Çapraz Konuşma" daha az sorun çıkarır, çünkü paralel iletişim kablolarına göre daha az iletken vardır.
 - Birçok entegre ve çevresel aygıtın seri arabirimleri vardır.
 - Farklı kanallar arasındaki saat eğriliği (clock skew) sorun değildir.
 - Uygulaması daha ucuzdur.
- **Saat Eğriliği (Clock Skew):** İki saat sinyali arasındaki gecikme.
- **Seri Veri İletim Modları:** Seri veri iletimi için, üç çalışma modu kullanılabilir.
 - **Simpleks (Tek Yönlü):** Basit bir bağlantıda, veriler yalnızca bir yönde iletilir. Örneğin, durum sinyallerini bilgisayara geri gönderemeyen bir bilgisayardan yazıcıya.
 - **Yarı Çift Yönlü:** Yarı çift yönlü bir bağlantıda, iki yönlü veri aktarımı mümkündür, ancak her seferinde yalnızca bir yönde veri iletilir.
 - **Tam Çift Yönlü:** Tam çift yönlü bir yapılandırmada, her iki uç da aynı anda veri gönderebilir ve alabilir.
- **İki cihaz birbiriyle Seri Haberleşme yaparken dikkat edilmesi gereken parametreler:**
 1. **Durdurma biti** uzunluğu (Eşzamansız mod)
 2. **Karakter** uzunluğu
 3. **Eşlik biti** (Parite biti)
 4. **Baud hızı** faktörü
- **Makro:** Programın başlangıcında bir isim verdiğimiz talimat grubudur. Kullanıldığı her yerde derleme aşamasında kod olarak eklenir.
- **Makro ve Prosedürlerin Karşılaştırılması**
 - Prosedürleri kullanmanın büyük bir avantajı, bir çok kez ihtiyaç duyulan makine kodlarının sadece bir kez ana belleğe yüklenmesinin gerekmesidir.
 - Prosedürleri kullanmanın dezavantajı, yığına (Stack) kullanımına ihtiyaç duymasındır.
 - Makro kullanmak, bir prosedürün çağırılması ve geri dönme ile ilgili ek zaman kaybını önler.
 - Makrolar, yalnızca kodunuz derlenene kadar var olur. Derlemeden sonra tüm makrolar gerçek komutlarla değiştirilir. Bir makro tanımladıysanız ve kodunuzda hiç kullanmadıysanız, derleyici onu görmezden gelir.
 - Alt programlar (Prosedürler) için **CALL** komutu kullanılır. Makroları çağırarak için, makro ismini kullanmak yeterlidir.

MyMacro MACRO p1, p2, p3

```
MOV AX, p1
MOV BX, p2
MOV CX, p3
```

ENDM

ORG 100h

MyMacro 1, 2, 3

MyMacro 4, 5, DX

RET

Yandaki kod adım adım işlendiğinde, aşağıdaki gibi çalışır.

MOV AX, 00001h ; MyMacro 1, 2, 3

MOV BX, 00002h

MOV CX, 00003h

MOV AX, 00004h ; MyMacro 4, 5, DX

MOV BX, 00005h

MOV CX, DX

- **Makrolarda LOCAL kullanımı:** Makro içerisinde etiket kullanılırsa, makro dışındaki etiketlerle aynı olduğunda, "duplicate declaration" hatası meydana gelir. Bunu önlemek için, makro içerisindeki etiketleri belirtmek için **LOCAL** komutu kullanılır.

- **Assembly dili, C dili içerisinde neden kullanılır?**

- Tüm CPU özellikleri, C dilinde erişime açık değildir. Özellikle sürücü ve işletim sistemi programlamasında, başka türlü erişimi mevcut olmayan özel Register'lara ve/veya komutlara erişilmesi gerektiğinde kullanılabilir.
- Kodun çakılması sonrasında hata ayıklama için kullanılabilir. Özellikle çökme noktasında, mümkün olduğu kadar çok program durumunu (yani tüm CPU Register vb. bilgilerini) yakalamak istediğimizde, Assembly dili C'den çok daha iyi bir araçtır.
- Daha etkin ve hızlı bir kod oluşturulmak istendiğinde kullanılabilir. Örneğin bir mikrosaniye içinde birçok kez tekrarlanan programın bir parçası gibi.

- **Assembly kodunun C dili içerisindeki kullanımı**

```
void main(void)
{
    _asm
    {
        mov     ah,8           ;read key no echo
        int     21h
        cmp     al,'0'        ;filter key code
        jnb     big
        cmp     al,'9'
        ja      big
        mov     dl,al         ;echo 0 - 9

        mov     ah,2
        int     21h

    big:
    }
}
```

Assembly dilinin kodları, **asm { ... }** blokları içerisinde yazılmalıdır.

Assembly komutları, **mutlaka küçük harfle** yazılmalıdır.

C++'da **asm (" ... ");** veya **__asm__ (" ... ");** şeklinde kullanılır.

- **Assembly dilini C dili içinde kullanırken dikkat edilmesi gerekenler:**

- 16 bit işletim sistemlerinde kullanılacak şekilde geliştirilen C derleyicileri, DOS işletim sistemi gözetilerek oluşturulmuştur. 32 bitlik işletim sistemleri için geliştirilen derleyiciler için ise bu söz konusu değildir. Bu nedenle DOS'ta geçerli interruptları kullanımda problemler çıkabilir.
- Portlara (seri, paralel, usb vb) erişimde Windows 95 ve 98'de doğrudan erişim varken (Bknz: IN ve OUT komut kullanımı) sonraki işletim sistemlerinde portları kullanmak istediğinizde KERNEL yazılması gerekmektedir.

- **Ekran Modları:** EMU8086'da yazacağımız kodlarda, ekranı kullanmadan önce, ekran modunu ayarlamamız gerekir.

- **Text (Metin) Ekran Modu:** Bu modda ekrana karakter yazdırmak için üç farklı yöntem kullanılır.
 - **MS-DOS:** INT 21h kesmesi kullanılır.
 - **BIOS:** Karakterler, BIOS hizmetleri olarak bilinen INT 10h işlevi kullanılarak çıkarılır. Bu INT 21h'den daha hızlı yürütülür ve metin renginin kontrolüne izin verir.
 - **Doğrudan Video Erişimi:** Karakterler doğrudan video RAM'e (ekran arabelleğine) taşınır, bu nedenle yürütme anlıktır.
- **Grafik Ekran Modu:** Piksel olarak kullanılır.

- **Ekran Ara (Tampon) Belleği:** Bu bellek, **B800:0000**'da başlar. Satır ve sütunları vardır. Sütun sayısı 80, satır sayısı ise 40 veya 25 olabilir. Sayfa numaraları 8'e kadardır. Default video sayfası, Sayfa 0'dır. Ekrandaki her sütun 160 bayt alır. (**Character + Attribute = 2 bayt x 80**)

BIT COLOR TABLE (STİL TABLOSU)					
HEX	BIN	COLOR	HEX	BIN	COLOR
0	0000	siyah	8	1000	koyu gri
1	0001	mavi	9	1001	açık mavi
2	0010	yeşil	A	1010	açık yeşil
3	0011	camgöbeği	B	1011	açık camgöbeği
4	0100	kırmızı	C	1100	açık kırmızı
5	0101	eflatun	D	1101	açık eflatun
6	0110	kahverengi	E	1110	sarı
7	0111	açık gri	F	1111	beyaz

Yüksek değerli 4 bit arka plan rengini, düşük değerli 4 bit yazı rengini verir.
(11110000b beyaz üzerine siyah yazı)